

Computability: Recitation 5

May 4, 2026

1 Computability Classes: R, RE

1.1 Reminder: definitions

Definition 1 (Language of a TM). *The language of a Turing machine M is*

$$L(M) := \{w \in \Sigma^* : M \text{ accepts } w\}.$$

Definition 2 (Recognizes). *A Turing machine M recognizes a language L if $L(M) = L$.*

Definition 3 (Decides). *A Turing machine M decides a language L if $L(M) = L$ and M halts on every input.*

Notice that if M decides L , M halts and rejects every $w \notin L$, but if M only recognizes L , M may also not halt on such words.

Definition 4 (RE). *RE is the set of all recognizable languages, that is, $\text{RE} := \{L : \text{There is a TM that recognizes } L\}$.*

Definition 5 (R). *R is the set of all decidable languages, that is, $\text{R} := \{L : \text{There is a TM that decides } L\}$.*

1.2 Time-bounded acceptance is decidable

We can use the universal Turing machine to recognize acceptance of a word by a Turing machine. Simulate the run of M on w , and accept if M accepts. Therefore, $A_{TM} := \{\langle M, w \rangle : M \text{ accepts } w\} \in \text{RE}$. However, as we will see later A_{TM} is not decidable. We show that a similar language that bounds the length of the run is decidable.

Proposition 6.

$$L := \{\langle M, w, k \rangle : M \text{ accepts } w \text{ within at most } k \text{ steps}\} \in \text{R}.$$

Proof. Construction: We construct a TM T that decides L . It operates as follows on input $\langle M, w, k \rangle$:

1. Simulate the run of M on w for at most k steps (by using a universal machine with a counter that counts steps on the side).
2. If M accepts w after at most k steps, T accepts.
3. Otherwise, T rejects.

Correctness:

- By the definition of T , it accepts a word $\langle M, w, k \rangle$ if and only if the run of M on w accepts within at most k steps, so it decides L .
- T halts on all inputs: it simulates the run for some finite number of steps k , so it doesn't enter an infinite loop.

□

1.3 NON-EMPTY_{TM} is recognizable

We show that the language of (the encodings of) Turing machines with a non-empty language is recognizable, later in the recitation we show it is not decidable.

Proposition 7.

$$\text{NON-EMPTY}_{TM} := \{\langle M \rangle : L(M) \neq \emptyset\} \in \text{RE}$$

Proof. Construction: We construct a TM T that recognizes NON-EMPTY_{TM} , it operates as follows on input $\langle M \rangle$:

For every $n = 0, 1, 2, \dots$:

For every $w \in \Sigma^*$ such that $|w| \leq n$:

Run M on w for n steps. If M reaches an accepting state, accept.

Correctness: If $\langle M \rangle \in L(T)$, by the construction, there exists $w \in \Sigma^*$ that M accepts, so $\langle M \rangle \in \text{NON-EMPTY}_{TM}$. If $\langle M \rangle \in \text{NON-EMPTY}_{TM}$, there exists a word w that M accepts. Denote by $k \in \mathbb{N}$ the length of the run of M on w . By the construction, in the $i := \max(|w|, k)$ -th iteration, T simulates the run of M on w (because $|w| \leq i$) for at least k steps (because $i \geq k$), and hence M reaches an accepting state, and T accepts. $\square$

1.4 Closure Properties of R and RE

1.4.1 Closure Properties of R

Proposition 8. *If $L \in R$ then $\bar{L} \in R$.*

Proof. Construction: Since $L \in R$, there exists a TM M that decides L . We use M to construct a new machine T that decides $\bar{L}$ by swapping the states q_{acc} and q_{rej} .

Correctness: First, note that since M halts on all inputs, the machine T also halts on all inputs, as the only difference between them is in the final states. $L(T) = \bar{L}$ since

$$T \text{ accepts } w \iff M \text{ rejects } w \iff w \notin L \iff w \in \bar{L}.$$

$\square$

Proposition 9. *If $L_1, L_2 \in R$ then $L_1 \cup L_2 \in R$.*

Proof. Construction: Since $L_1, L_2 \in R$, there exist TMs M_1, M_2 that decide them. We define a new TM T that decides $L_1 \cup L_2$. It operates as follows:

1. Run M_1 on w . If it accepts, accept.
2. Run M_2 on w . If it accepts, accept.
3. Otherwise, reject.

Remark: To accomplish this, T can duplicate the word w to a more distant location on the tape and mark it with a special symbol. Whenever M_1 's computation reaches this symbol, T uses a procedure to move w even further along the tape. Upon M_1 's termination, if acceptance does not occur, T clears the tape used by M_1 , and repositions the head to the beginning of the duplicated copy of w and initiates the computation of M_2 .

Correctness:

- Since M_1, M_2 halt on all inputs, T also halts on all inputs.
- By the construction, it is clear that T accepts a word if and only if M_1 accepts it or M_2 accepts it, so $L(T) = L_1 \cup L_2$.

$\square$

Proposition 10. *If $L_1, L_2 \in R$ then $L_1 \cap L_2 \in R$.*

We can show it by constructing a Turing machine that operates like the machine used in Proposition 9, with the difference that it accepts only if both machines accept.

Remark: It also follows from closure to complement and union since $L_1 \cap L_2 = \overline{\overline{L_1} \cup \overline{L_2}}$.

1.4.2 Closure Properties of RE

When showing closure properties of RE, we deal with Turing machines that may not halt.

Proposition 11. *If $L_1, L_2 \in RE$ then $L_1 \cap L_2 \in RE$.*

To show the closure of RE under intersection, the same construction used to show the closure of R to intersection works. If one of the machines does not halt, the constructed machine does not halt, which is the desired behavior: the word is not accepted by the non-halting machine, and hence is not in the intersection.

However, when constructing a TM for the union, one machine may not halt and prevent the constructed machine from seeing that the other one accepts, leading to non-halting even though the word is in the union. To solve this, we use an iterative technique to simulate both machines “in parallel”.

Proposition 12. *If $L_1, L_2 \in RE$ then $L_1 \cup L_2 \in RE$.*

Proof. Construction: Let M_1, M_2 be TMs that recognize L_1, L_2 respectively. We define a TM T that recognizes $L_1 \cup L_2$. It operates as follows on input w :

1. For every $n = 0, 1, 2, \dots$:
 - (a) Run M_1 on w for n steps. If M_1 reaches an accepting state, accept.
 - (b) Run M_2 on w for n steps. If M_2 reaches an accepting state, accept.

Correctness:

- If $w \in L_1 \cup L_2$, WLOG $w \in L_1$ (the second case is identical). Since $w \in L_1$, the machine M_1 necessarily accepts w within some finite number of steps k . In the k -th iteration, T simulates the run of M_1 on w for k steps, so T will accept w .
- If $w \in L(T)$, then by the definition of the machine’s operation, one of the machines M_1 or M_2 necessarily accepts w . Therefore, $w \in L_1 \cup L_2$.

In total, we got that $L(T) = L_1 \cup L_2$, and so by the definition of the class RE , we get that $L_1 \cup L_2 \in RE$. $\square$

Remark: We will later see that RE is not closed under complement.

2 Reductions

Definition 13 (Output of a TM). *If a TM M halts on w , $M(w)$ is the content of the tape at the end of its run (neglecting spaces).*

Definition 14 (Computable). *A function $f : \Sigma^* \rightarrow \Sigma^*$ is called computable if there exists a TM M_f that computes it: for every input $w \in \Sigma^*$, $M_f(w) = f(w)$.*

Definition 15 (Reduction). *A reduction from language $A \subseteq \Sigma^*$ to language $B \subseteq \Sigma^*$ is a computable function $f : \Sigma^* \rightarrow \Sigma^*$ such that for all $x \in \Sigma^*$:*

$$x \in A \Leftrightarrow f(x) \in B$$

If such a function f exists, we denote $A \leq_m B$.

A Turing machine that computes f is called a “reduction machine”.

2.1 Example: Parity Languages

Example 16. *Consider the following languages over $\Sigma = \{a, b\}$:*

$$L_1 = \{w \in \Sigma^* : |w| \bmod 2 = 0\} \text{ (even length strings)}$$

$$L_2 = \{w \in \Sigma^* : |w| \bmod 2 = 1\} \text{ (odd length strings)}$$

We claim that $L_1 \leq_m L_2$.

2.1.1 Reduction 1: solve the problem

Define $f : \Sigma^* \rightarrow \Sigma^*$ as follows:

$$f(w) = \begin{cases} a & \text{if } |w| \bmod 2 = 0 \\ aa & \text{if } |w| \bmod 2 = 1 \end{cases}$$

f is a computable function: we can compute the length of a word using a TM, and determine if it’s even or odd. Let’s prove that $x \in L_1 \Leftrightarrow f(x) \in L_2$ holds:

- If $w \in L_1$, then $|w| \bmod 2 = 0$. So f will return a - a word with an odd number of letters, and thus in L_2 .

- If $w \notin L_1$, then $|w| \bmod 2 \neq 0$, so f will return aa - a word with an even number of letters, and thus not in L_2 .

We found a computable function that satisfies $x \in L_1 \Leftrightarrow f(x) \in L_2$, and therefore by definition, $L_1 \leq_m L_2$.

2.1.2 Reduction 2: transfer the problem

Alternatively, define:

$$f(w) = a \cdot w \text{ (concatenate an 'a' in the beginning)}$$

f is a computable function: the TM that calculates f moves one cell to the left, writes an 'a' and end its run. Let's prove that $x \in L_1 \Leftrightarrow f(x) \in L_2$ holds:

- If $w \in L_1$, then $|w| \bmod 2 = 0$, and so: $|f(w)| \bmod 2 = |a \cdot w| \bmod 2 = (|w| + 1) \bmod 2 = 1$ From the definition of L_2 , we get that $a \cdot w \in L_2$.
- If $w \notin L_1$, then $|w| \bmod 2 = 1$, and so: $|f(w)| \bmod 2 = |a \cdot w| \bmod 2 = (|w| + 1) \bmod 2 = 0$ From the definition of L_2 , we get that $a \cdot w \notin L_2$.

Note: The reduction is computable in **constant** time!

Note: The first reduction decides whether $w \in L_1$ or not, and accordingly returns a different output. It is impossible if $L_1 \notin R$. In contrast, the second reduction cleverly maps the first problem to the second without solving it.

2.2 Example: NON-EMPTY_{TM}

Proposition 17.

$$\text{HALT}_{TM} \leq_m \text{NON-EMPTY}_{TM}$$

How to approach it?

When we are asked to show that $L_1 \leq_m L_2$ we should ask ourselves the following questions:

1. **What should we construct?** That one is easy - it is always a computable function $f : \Sigma^* \rightarrow \Sigma^*$.
2. **What is the type of the input and output of f ?** Of course, the input and the output can be any word, but since handling invalid inputs is usually an easy task, we focus on "valid" inputs. In our case, f takes an encoding of a TM and a word $\langle M, w \rangle$ as input and outputs an encoding of a TM $\langle T_{M,w} \rangle$.
3. **What condition should f satisfy?** By the definition of reduction and the definitions of the languages, M halts on w if and only if the language of T is not empty.

Let's put it together: We should construct a function f that takes a TM and a word $\langle M, w \rangle$ as input and outputs a TM $\langle T_{M,w} \rangle$ such that M halts on w if and only if the language of T is not empty.

After we understand what should we construct, we can focus on the real challenge - how to construct such a function?

Common Pitfall

It is tempting to suggest the following reduction:

$$f(\langle M, w \rangle) = \begin{cases} \langle M_{ALL} \rangle & \text{if } M \text{ halts on } w \\ \langle M_{EMPTY} \rangle & \text{otherwise} \end{cases}$$

where M_{ALL} is the TM that immediately accepts every input word x , and M_{EMPTY} is the machine that immediately rejects every input word x .

Clearly, the condition is satisfied. If M halts on w , the language of the output machine is not empty, and it does not halt, the language is empty.

So, what is the problem here? f is not computable. As we saw in class, there is no machine that checks whether a given machine M halts on a given input w .

This function is similar to the first reduction we saw in the previous example. The difference is that checking parity of the length is decidable, unlike checking if a machine halts on an input.

To show that f is a reduction, we should show that it is computable, namely that there exists a TM M_f that computes f . In our example, M_f cannot check if M halts on w , so probably M and w should be encoded in $T_{M,w} := f(\langle M, w \rangle)$ so as a part of its run on an input x it can simulate the run of M on w . the intuition is

that if M does not halt on w , T should not accept, no matter what is its input x , and if M halts on w it should accept some input, for simplicity we can design it to accept every input x .

Proof. Construction: Let $f : \Sigma^* \rightarrow \Sigma^*$ be the function that returns ϵ for “invalid input” (that is, input which is not an encoding of a TM and a word), and for valid input returns

$$f(\langle M, w \rangle) = \langle T_{M,w} \rangle$$

where $T_{M,w}$ operates as follows on input x : it deletes x from the tape, writes w on the tape, and runs M on w as a procedure. If M reaches a final state it accepts.

Computability: f is computable:

1. Checking the validity of the input (whether it is an encoding of a TM and a word) is computable.
2. Given a TM M and a word w constructing such a TM is computable by “combining” a TM that deletes its input, a TM that writes w on its tape, and M .

Correctness: For invalid input (which is not in HALT_{TM}) the reduction returns invalid input (which is not in NON-EMPTY_{TM}). For valid input:

- If $\langle M, w \rangle \in \text{HALT}_{TM}$, by definition M halts on w , and hence by the construction, $T_{M,w}$ accepts every input, so $T_{M,w} \in \text{NON-EMPTY}_{TM}$.
- If $\langle M, w \rangle \notin \text{HALT}_{TM}$, by definition M does not halt on w , and hence by the construction, $T_{M,w}$ does not halt and does not accept any input, so $\langle T_{M,w} \rangle \notin \text{NON-EMPTY}_{TM}$.

To sum up, f is a computable function such that $y \in \text{HALT}_{TM} \iff f(y) \in \text{NON-EMPTY}_{TM}$, so $\text{HALT}_{TM} \leq_m \text{NON-EMPTY}_{TM}$. $\square$

2.3 Reduction theorem

Theorem 18 (reduction theorem). *Let $L_1, L_2 \subseteq \Sigma^*$ be languages such that $L_1 \leq_m L_2$. If $L_2 \in R$, then $L_1 \in R$ (Equivalently, if $L_1 \notin R$ then $L_2 \notin R$). Similarly, If $L_2 \in \text{RE}$ then $L_1 \in \text{RE}$ (and equivalently, if $L_1 \notin \text{RE}$, then $L_2 \notin \text{RE}$).*

Intuition: If $L_1 \leq_m L_2$ then L_2 is “at least as hard” as L_1 . If there is something we know how to do with L_2 (decide it, or recognize it, or recognize its complement), we can do the same for L_1 by first applying the reduction. Equivalently, if there is something we know is impossible to do with L_1 , then the same applies for L_2 .

Example: By Proposition 17, $\text{HALT}_{TM} \leq_m \text{NON-EMPTY}_{TM}$, and we saw in class that $\text{HALT}_{TM} \notin R$, so by the reduction theorem also $\text{NON-EMPTY}_{TM} \notin R$.

3 Extra (if time permits): $\text{HALT}_{TM} \notin R$

Claim. There is no Turing machine that decides HALT_{TM} , that is, $\text{HALT}_{TM} \notin R$.

Proof. Assume, for the sake of contradiction, that there exists a Turing machine D that decides HALT_{TM} .

Construct a Turing machine E that, given an input $\langle M \rangle$, operates as follows:

1. E multiplies the input to get $\langle M, \langle M \rangle \rangle$.
2. E runs D on $\langle M, \langle M \rangle \rangle$.
3. If D returns “yes”, then E enters an infinite loop. If D returns “no”, then E halts (accepts or rejects).

Now, consider running E on its own description $\langle E \rangle$. Does E halt on input $\langle E \rangle$?

Case 1: Suppose yes, then $\langle E, \langle E \rangle \rangle \in \text{HALT}_{TM}$ so D should accept $\langle E, \langle E \rangle \rangle$. Then, by the definition of E , it enters an infinite loop and not halt - a contradiction.

Case 2: Suppose no, then $\langle E, \langle E \rangle \rangle \notin \text{HALT}_{TM}$ so D should reject $\langle E, \langle E \rangle \rangle$. Then, by the definition of E , it halts - a contradiction.

In both cases we reach a contradiction. Therefore, our assumption that such a Turing machine D exists must be false. Hence, HALT_{TM} is undecidable.